

Midterm Review

All About the Midterm

The midterm will be held this **Thursday 4/24/2025 from 6-8pm** in **Kent 107**. The exam period will be **120 minutes** long. Outside resources will **not be allowed** during the exam, except for a single, hand-written, double sided, letter sized cheatsheet. Any content covered in class up to **Friday's lecture (4/18)** may be present on the exam. The following lists some of the topics that might be covered on the midterm. The brackets contain relevant chapter numbers in DPV.

1. Algorithmic Analysis Principles

- Big-O Notation [0.3]
- Induction and Strong Induction
- Addition [1.1, 1.2], Multiplication[2.1]
- Prime Number Testing [1.3]

2. Divide and Conquer [2]

- General algorithm structure
- Proof of correctness
- Runtime analysis
 - Master Theorem [2.2]
 - Analysis through summations [2.2]
- Multiplication [2.1]
- Mergesort [2.3]
- Quickselect [2.4]
- FFT [2.6]

3. Graphs [3, 4]

- Representations: adjacency matrix, adjacency list
- Graph Traversals
 - Depth-First Search [3.2, 3.3, 4.1]
 - * Output Tree + Types of Edges
 - * Pre-ordering and Post-ordering [3.2]
 - * On Directed Graphs + DAGs [3.3]

Tips on Studying

Here are some studying tips for Thursday's midterm.

- Review the course lecture notes, everything covered up to Friday's (4/18) lecture is fair game for the midterm.
- Review the course textbook *Dasgupta, Papadimitriou & Vazirani*. The course lecture notes also follow the textbook rather closely.
- Review your previous homeworks. If you would like more feedback on how to answer a question, stop by office hours or post on EdStem.
- Review previous discussion section worksheets. They are a good source of practice problems.
- Review your notes, and discuss your understanding with friends / course partners. Explaining how you understand an idea to someone else is the best way to test your knowledge.
- Practice with problems in this packet, as well as the exercises in DPV.
- Most importantly, make sure to get a good night's rest before the exam on Thursday!

PROBLEM 1 (Big-Order). Order the functions so that f_i comes before f_j if and only if $f_i(n) \leq O(f_j(n))$.

(1) $f_1(n) = 3^n$

(2) $f_2(n) = n^{1/3}$

(3) $f_3(n) = 12$

(4) $f_4(n) = 2^{\log_2(n)}$

(5) $f_5(n) = \sqrt{n}$

(6) $f_6(n) = 2^n$

(7) $f_7(n) = \log_2(n)$

(8) $f_8(n) = 2^{\sqrt{n}}$

(9) $f_9(n) = n^3$

PROBLEM 2 (Super Polynomial). In this question, we'll see whether or not there is a function whose growth-rate is strictly between polynomials, and exponentials.

Prove or disprove the following claim: if $f : \mathbb{N} \rightarrow \mathbb{N}$ is any positive-valued function, then either (1) there exists a constant $c > 0$ such that $f(n) \leq O(n^c)$, or (2) there exists a constant $\alpha > 1$ such that $f(n) \geq \Omega(\alpha^n)$.

PROBLEM 3 (Master Theorem). Let's review how to use the master theorem to solve recurrence relations.

(a) Complete the following statement of the Master theorem.

If $T(n) = a \cdot T(\lceil n/b \rceil) + O(n^d)$ for some constants $a > 0$, $b > 1$, and $d \geq 0$, then...

(b) Solve the following recurrence relations

i. $T(n) = 4 \cdot T(n/2) + 42 \cdot n$

ii. $T(n) = 4 \cdot T(n/3) + n^2$

iii. $T(n) = 8 \cdot T(n/4) + 18 \cdot n^{3/2}$

PROBLEM 4. Let's solve more recurrence relations.

(a) $T(n) = T(n/2) + n \cdot \log n$

(b) $T(n) = T(n-1) + n^c$ where $c > 0$ is some constant

(c) $T(n) = T(n-1) + \alpha^n$ where $\alpha > 1$ is some constant

(d) $T(n) = 4 \cdot T(n-4) + 1$

PROBLEM 5 (Hadamard Matrices). The Hadamard matrices are a sequence of matrices $\mathbf{H}_0, \mathbf{H}_1, \dots$ defined recursively as follows: $\mathbf{H}_0 = \begin{pmatrix} 1 \end{pmatrix}$, and for all $k > 0$, $\mathbf{H}_k$ is the $2^k \times 2^k$ matrix

$$\mathbf{H}_k = \begin{pmatrix} \mathbf{H}_{k-1} & \mathbf{H}_{k-1} \\ \mathbf{H}_{k-1} & -\mathbf{H}_{k-1} \end{pmatrix}$$

(a) Write down the Hadamard matrices $\mathbf{H}_0$, $\mathbf{H}_1$, and $\mathbf{H}_2$

(b) Using $\mathbf{H}_2$, compute the matrix vector product $\mathbf{H}_2 \mathbf{v}$ where $\mathbf{v} \in \mathbb{R}^4$ is the column vector

$$\mathbf{v} = \begin{pmatrix} 1 \\ -1 \\ -1 \\ 1 \end{pmatrix}$$

Note that since $\mathbf{H}_2$ is a 4×4 matrix, and $\mathbf{v}$ is a 4-dimensional vector, the resulting vector should also be 4-dimensional.

(c) Let us now denote $\mathbf{v}_1$ and $\mathbf{v}_2$ as the top and bottom halves of $\mathbf{v}$ respectively. That is define the vectors $\mathbf{v}_1 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$, $\mathbf{v}_2 = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$, so that $\mathbf{v} = \begin{pmatrix} \mathbf{v}_1 \\ \mathbf{v}_2 \end{pmatrix}$. Compute $\mathbf{u}_1 = \mathbf{H}_1(\mathbf{v}_1 + \mathbf{v}_2)$ and $\mathbf{u}_2 = \mathbf{H}_1(\mathbf{v}_1 - \mathbf{v}_2)$ to get two vectors of length 2.

(d) Stack $\mathbf{u}_1$ and $\mathbf{u}_2$ to get a vector $\mathbf{u}$ of length 4. What do you notice about $\mathbf{u}$?

- (e) Suppose that $\mathbf{v} = \begin{pmatrix} \mathbf{v}_1 \\ \mathbf{v}_2 \end{pmatrix}$ is a column vector of length $n = 2^k$. Additionally, let $\mathbf{v}_1$ and $\mathbf{v}_2$ be the top and bottom half respectively, so that both have length $\frac{n}{2} = 2^{k-1}$. Write the matrix vector product $\mathbf{H}_k \mathbf{v}$ in terms of $\mathbf{H}_{k-1}$, $\mathbf{v}_1$ and $\mathbf{v}_2$ (note that $\mathbf{H}_{k-1}$ is a $\frac{n}{2} \times \frac{n}{2}$ dimensional matrix, or $2^{k-1} \times 2^{k-1}$).
- (f) Use the results from part (d) and part (e) to come up with a divide-and-conquer algorithm to compute the matrix vector product $\mathbf{H}_k \mathbf{v}$. Write pseudocode for your algorithm, then show that it can compute the matrix-vector in $O(n \log n)$ operations where $n = 2^k$. Assume that basic arithmetic operations like addition and multiplication take $O(1)$ time.

PROBLEM 6 (The FUN Fourier Transform). In this question, we'll practice using FFT. Some exercises are taken from DPV 2.8 and 2.9.

(a) What is the FFT of $\begin{pmatrix} 1 & 0 & 0 & 0 \end{pmatrix}$, and what is the appropriate value of ω in this case?

(b) For what coefficient vector is $\begin{pmatrix} 1 & 0 & 0 & 0 \end{pmatrix}$ the FFT?

(c) What is the degree-3 polynomial $p(x)$ whose evaluation on the 4-th roots of unity are

$$p(1) = 10 \quad p(i) = 4 - 2i \quad p(-1) = 10 \quad p(-i) = 4 + 2i$$

(d) Multiply $x + 1$ and $x^2 + 1$ using FFT. Precisely, choose an appropriate power of two, find the FFT of the two sequences, multiply the results componentwise and compute the inverse FFT to get the result.

(e) Repeat the task for the pair of polynomials $1 + x + 2x^2$ and $2 + 3x$.

PROBLEM 7 (Adapted from Kleinberg-Tardos Chapter 5 - Exercise 6). The following problems are about finding local minima in trees. Consider a tree T . Each node v of T is labeled with a real number x_v . You may assume that the real numbers labeling the nodes are all distinct. A node v of T is a local minimum if the label x_v is less than the label x_w for all nodes w that are joined to v by an edge.

- (a) Suppose T is an n -node complete binary tree T , where $n = 2^d - 1$ for some d . You are given such a complete binary tree T , but the labeling is only specified in the following implicit way: for each node v , you can determine the value x_v by probing the node v . Show how to find a local minimum of T using only $O(\log n)$ probes to the nodes of T .

- (b) Give a proof sketch of the following statement: every tree T contains a vertex v^* , such that when v^* is removed, every connected component of the graph $T \setminus \{v^*\}$ contains at most $n/2$ vertices.

- (c) Suppose now that T is an arbitrary tree, and that you have access to an oracle that on input $v \in T$ returns a tuple (x_v, w, x_w) where w is the neighbor of v in T minimizing x_w . Design an algorithm that finds a local minimum of T by querying the oracle at most $O(\log n)$ times.

PROBLEM 8 (DFS). Consider the following directed acyclic graph (DAG):

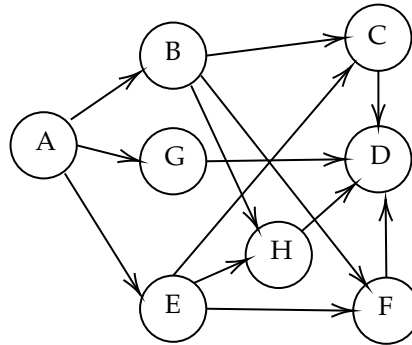


Figure 1: The graph G

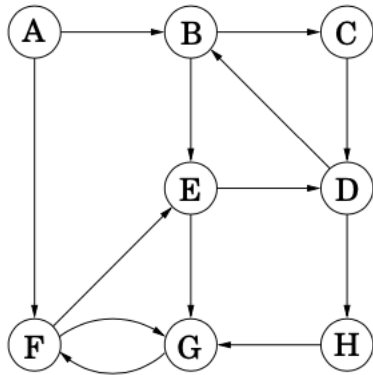
- (a) How many distinct topological orderings of G are there?
- (b) Suppose you run depth-first-search on G starting at vertex A , and that the algorithm explores the vertices in alphabetical order. During the algorithm, the postvisit procedure labels the vertex with the current value of the clock as in the textbook. At the end of the algorithm, you sort the vertices by increasing value of $post(v)$. What is the resulting ordering of the vertices?

PROBLEM 9 (DAGs and SCCs). Prove or disprove the following statement:

“A directed acyclic graph (DAG) with n vertices always has n strongly connected components.”

PROBLEM 10 (Problem 3.2 from DPV). Perform depth-first search on each of the following graphs; whenever there's a choice of vertices, pick the one that is alphabetically first. Classify each edge as a tree edge, forward edge, back edge, or cross edge, and give the pre and post number of each vertex.

(a)



(b)

